МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

Ордена Трудового Красного Знамени федеральное государственное бюджетное образовательное учреждение высшего образования

«Московский технический университет связи и информатики»

Кафедра «Программная инженерия»

Дисциплина «Структуры и алгоритмы обработки данных»

Отчет по лабораторной работе №2

Выполнила:

Студент группы БВТ2504
Мясникова Е.А.

Проверила: Шедова А.Р

Москва, 2026 г.

Содержание

Цель работы:.....2
Ход выполненной работы:.....2
1. Подготовка.....2
2. Задание 1.....2
3. Задание 2.....5
4. Задание 3.....10
5. Задание 4.....13
6. Задание 5.....13
7. Задание 6.....19
Заключение.....20

Цель работы: Освоить рекурсию как метод построения алгоритмов и реализовать базовые линейные структуры данных, подтвердив их асимптотические свойства (включая амортизированную сложность) экспериментально.

Ход выполненной работы:

1. **Подготовка**

В личном репозитории создан каталог student_work/lab02. Стартовая заготовка скопирована в него под именем lab02.py. Проверена версия Python 3.13.5. Заготовка сохранена коммитом 52799af и отправлена на GitHub. Для работы используется вариант 16, seed = 46.

```
elena@tuf:~/Second-year-bachelor's-labs/data-structures-and-algorithms-labs$ python3 --version
Python 3.13.5
elena@tuf:~/Second-year-bachelor's-labs/data-structures-and-algorithms-labs$ ls -l student_work/lab02/lab02.py
-rw-rw-r-- 1 elena elena 13542 окт 12 22:55 student_work/lab02/lab02.py
elena@tuf:~/Second-year-bachelor's-labs/data-structures-and-algorithms-labs$ git log -1 --oneline
52799af (HEAD -> main, origin/main) Add lab 02 starter
elena@tuf:~/Second-year-bachelor's-labs/data-structures-and-algorithms-labs$ git status
Текущая ветка: main
Эта ветка соответствует «origin/main».

нечего коммитить, нет изменений в рабочем каталоге
elena@tuf:~/Second-year-bachelor's-labs/data-structures-and-algorithms-labs$
```

Рисунок 1 - Подготовка рабочей папки и сохранение заготовки в репозитории

Вывод: Рабочая папка и стартовый файл подготовлены. Версия Python соответствует требованию 3.10+, начальное состояние работы сохранено в репозитории.

2. **Задание 1**

Реализованы рекурсивные функции вычисления факториала, чисел Фибоначчи наивным способом и с мемоизацией, а также количества перемещений в задаче о Ханойских башнях. Для каждой функции предусмотрены базовые условия и обработка отрицательного аргумента. Проверки сохранены в отдельном файле check_recursion.py. Факториал сверён с math.factorial для значений от 0 до 100. Для Фибоначчи проверены восемь значений n и сравнены счётчики вызовов. Для Ханойских башен при n от 0 до 12 проверено совпадение числа перемещений с формулой $2^n - 1$.

```python
def factorial(n: int) -> int:  3 usages  Lena *
    """Рекурсивный факториал: O(n) вызовов и O(n) памяти стека."""
    if n < 0:
        raise ValueError("n должно быть неотрицательным")
    if n <= 1:
        return 1
    return n * factorial(n - 1)
```

Рисунок 2 - Код рекурсивной функции вычисления факториала

```
0! = 1
1! = 1
5! = 120
10! = 3628800
Сравнение с math.factorial для чисел от 0 до 10 — OK
Отрицательное число: ValueError — OK
```

Рисунок 3 - Проверка факториала (№1.1)

```python
def fib_naive(n: int) -> int:  3 usages  Lena *
    """Фибоначчи наивной рекурсией: экспоненциальное число вызовов."""
    if n < 0:
        raise ValueError("n должно быть неотрицательным")
    CALLS["fib_naive"] += 1
    if n <= 1:
        return n
    return fib_naive(n - 1) + fib_naive(n - 2)


def fib_memo(n: int, memo: dict[int, int] | None = None) -> int:  3 usages  Lena *
    """Фибоначчи с мемоизацией: O(n) вызовов при пустом кэше."""
    if n < 0:
        raise ValueError("n должно быть неотрицательным")
    CALLS["fib_memo"] += 1
    if n <= 1:
        return n
    if memo is None:
        memo = {}
    if n not in memo:
        memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
    return memo[n]
```

Рисунок 4 - Реализация вычисления чисел фибоначчи и с мемоизацией

```
Числа Фибоначчи и количество вызовов

 n |   F(n) |         Наивная |   С мемоизацией
-------------------------------------------------
 0 |      0 |               1 |               1
 1 |      1 |               1 |               1
 2 |      1 |               3 |               3
 5 |      5 |              15 |               9
10 |     55 |             177 |              19
15 |    610 |            1973 |              29
20 |   6765 |           21891 |              39
25 |  75025 |          242785 |              49

Результаты обеих функций — OK
Отрицательный аргумент: ValueError — OK
```

Рисунок 5 - Проверка чисел фибоначчи и сравнение количества вызовов наивной рекурсии и рекурсии с мемоизацией (№1.2)

```python
def hanoi(n: int, src: str = "A", dst: str = "C", aux: str = "B") -> int:  3 usages  Lena *
    """Вернуть число перемещений; O(2**n) вызовов, O(n) глубина стека."""
    if n < 0:
        raise ValueError("n должно быть неотрицательным")
    if n == 0:
        return 0
    return hanoi(n - 1, src, aux, dst) + 1 + hanoi(n - 1, aux, dst, src)
```

Рисунок 6 - Реализация рекурсивной функции подсчёта перемещений в задаче о ханойских башнях (№1.3)

```
factorial: OK

 n |   F(n) | Наивная | Мемоизация
 0 |      0 |       1 |          1
 1 |      1 |       1 |          1
 2 |      1 |       3 |          3
 5 |      5 |      15 |          9
10 |     55 |     177 |         19
15 |    610 |    1973 |         29
20 |   6765 |   21891 |         39
25 |  75025 |  242785 |         49
Фибоначчи: OK

 n | Перемещения | 2**n - 1
 0 |           0 |        0
 1 |           1 |        1
 2 |           3 |        3
 3 |           7 |        7
 4 |          15 |       15
 5 |          31 |       31
 6 |          63 |       63
 7 |         127 |      127
 8 |         255 |      255
 9 |         511 |      511
10 |        1023 |     1023
11 |        2047 |     2047
12 |        4095 |     4095
Ханойские башни: OK

Process finished with exit code 0
```

Рисунок 7 - Результаты проверки факториала, чисел фибоначчи и ханойских башен (все вместе)

Вывод: Все проверки пройдены. При n = 25 обе версии Фибоначчи получили 75025, но наивная версия выполнила 242785 вызовов, а версия с мемоизацией 49. Мемоизация сокращает число вызовов с экспоненциального до линейного за счёт сохранения

промежуточных результатов. Для 12 дисков получено 4095 перемещений, что соответствует формуле $2^n - 1$. Базовые условия обеспечивают завершение рекурсии.

3. Задание 2

Был реализован динамический массив с добавлением, чтением и изменением элементов. Когда свободные ячейки заканчивались, ёмкость увеличивалась вдвое, а элементы переносились в новый буфер. Работа массива была проверена сравнением с обычным списком Python, включая проверку недопустимых индексов. Затем для шести размеров массива было измерено время добавления элементов: после прогрева выполнялось пять повторов и выбиралась медиана. Результаты были сохранены в таблицу и представлены на графике.

```python
class DynamicArray:  3 usages  Lena *
    """Динамический массив с удвоением ёмкости."""

    INITIAL_CAPACITY = 4

    def __init__(self) -> None:  Lena *
        self._size = 0
        self._capacity = self.INITIAL_CAPACITY
        self._buffer: list = [None] * self._capacity

    def __len__(self) -> int:  Lena
        return self._size

    @property  1 usage  Lena
    def capacity(self) -> int:
        return self._capacity

    def _grow(self) -> None:  1 usage  Lena *
        new_capacity = self._capacity * 2
        new_buffer = [None] * new_capacity
        for i in range(self._size):
            new_buffer[i] = self._buffer[i]
        self._buffer = new_buffer
        self._capacity = new_capacity

    def append(self, value) -> None:  Lena *
        if self._size == self._capacity:
            self._grow()
        self._buffer[self._size] = value
        self._size += 1

    def get(self, index: int):  Lena *
        if index < 0 or index >= self._size:
            raise IndexError("Индекс вне границ массива")
        return self._buffer[index]

    def set(self, index: int, value) -> None:  Lena *
        if index < 0 or index >= self._size:
            raise IndexError("Индекс вне границ массива")
        self._buffer[index] = value
```

Рисунок 8 - Код динамического массива с удвоением емкости

```
Пустой массив: OK

Размер | Ёмкость
     0 |        4
     1 |        4
     4 |        4
     5 |        8
     8 |        8
     9 |       16
    16 |       16
    17 |       32
    32 |       32
    33 |       64
    64 |       64
    65 |      128
   100 |      128

100 добавлений и 100 изменений: совпадение с list - OK
Недопустимые индексы: IndexError - OK
DynamicArray: OK

Process finished with exit code 0
```

Рисунок 9 - Проверка динамического массива

```
def benchmark_append(seed: int) -> None:  new *
    """Измерить время append и сохранить результаты и график."""
    import csv
    import json
    import platform
    from pathlib import Path

    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    sizes = [1_000, 3_000, 10_000, 30_000, 100_000, 300_000]
    repeats = 5

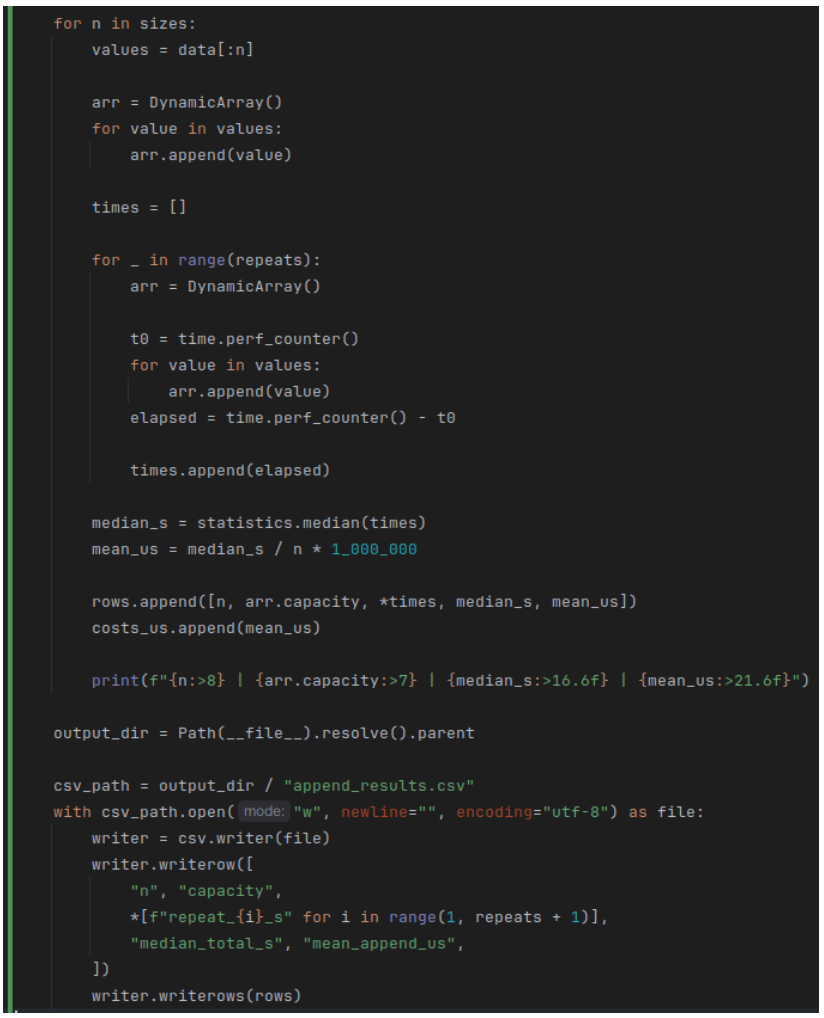
    rng = random.Random(seed)
    data = [rng.randint(-1_000_000,  b: 1_000_000) for _ in range(max(sizes))]

    cpu = "Не определён"
    for line in Path("/proc/cpuinfo").read_text(encoding="utf-8").splitlines():
        if line.startswith("model name"):
            cpu = line.split( sep: ":",  maxsplit: 1)[1].strip()
            break

    print(f"\nCPU: {cpu}")
    print(f"Python: {platform.python_version()}")
    print(f"Seed: {seed}; прогрев: 1; повторов: {repeats}; обработка: медиана")
    print("\n       n | Ёмкость | Медиана серии, с | Среднее на append, мкс")

    rows = []
    costs_us = []
```

Рисунок 10 - Замер времени добавления элементов в динамический массив, прогрев и пять повторов



```
    for n in sizes:
        values = data[:n]

        arr = DynamicArray()
        for value in values:
            arr.append(value)

        times = []

        for _ in range(repeats):
            arr = DynamicArray()

            t0 = time.perf_counter()
            for value in values:
                arr.append(value)
            elapsed = time.perf_counter() - t0

            times.append(elapsed)

        median_s = statistics.median(times)
        mean_us = median_s / n * 1_000_000

        rows.append([n, arr.capacity, *times, median_s, mean_us])
        costs_us.append(mean_us)

        print(f"{n:>8} | {arr.capacity:>7} | {median_s:>16.6f} | {mean_us:>21.6f}")

    output_dir = Path(__file__).resolve().parent

    csv_path = output_dir / "append_results.csv"
    with csv_path.open( mode: "w", newline="", encoding="utf-8") as file:
        writer = csv.writer(file)
        writer.writerow([
            "n", "capacity",
            *[f"repeat_{i}_s" for i in range(1, repeats + 1)],
            "median_total_s", "mean_append_us",
        ])
        writer.writerows(rows)
```

Рисунок 11 - Замер времени добавления элементов в динамический массив, прогрев и пять повторов

```python
metadata = {
    "cpu": cpu,
    "python": platform.python_version(),
    "platform": platform.platform(),
    "seed": seed,
    "sizes": sizes,
    "warmups_per_size": 1,
    "repeats_per_size": repeats,
    "timer": "time.perf_counter",
    "processing": "median_total_s / n * 1e6",
    "fresh_array_each_run": True,
    "data_generated_before_timing": True,
    "total_time_unit": "seconds",
    "append_time_unit": "microseconds",
}

metadata_path = output_dir / "append_environment.json"
metadata_path.write_text(
    json.dumps(metadata, ensure_ascii=False, indent=2) + "\n",
    encoding="utf-8",
)

# Строим график с логарифмическими осями.
fig, ax = plt.subplots(figsize=(8, 5))
ax.loglog( *args: sizes, costs_us, "o-", label="Медиана времени серии / n")
ax.set_xlabel("Количество добавлений n (логарифмическая шкала)")
ax.set_ylabel("Среднее время append, мкс (логарифмическая шкала)")
ax.set_title(f"Средняя стоимость append, seed = {seed}")
ax.grid( visible: True, which="both", alpha=0.3)
ax.legend()
fig.tight_layout()

graph_path = output_dir / "append_benchmark.png"
fig.savefig(graph_path, dpi=200)
plt.close(fig)

print("\nСохранены файлы:")
for path in (csv_path, metadata_path, graph_path):
    print(path)
```

Рисунок 12 - Замер времени добавления элементов в динамический массив, прогрев и пять повторов

```
{
  "cpu": "Intel(R) Core(TM) i7-14650HX",
  "python": "3.13.5",
  "platform": "Linux-6.12.111+deb13-amd64-x86_64-with-glibc2.41",
  "seed": 46,
  "sizes": [
    1000,
    3000,
    10000,
    30000,
    100000,
    300000
  ],
  "warmups_per_size": 1,
  "repeats_per_size": 5,
  "timer": "time.perf_counter",
  "processing": "median_total_s / n * 1e6",
  "fresh_array_each_run": true,
  "data_generated_before_timing": true,
  "total_time_unit": "seconds",
  "append_time_unit": "microseconds"
}
```


Рисунок 13 - Результаты измерения средней стоимости append для шести размеров массива

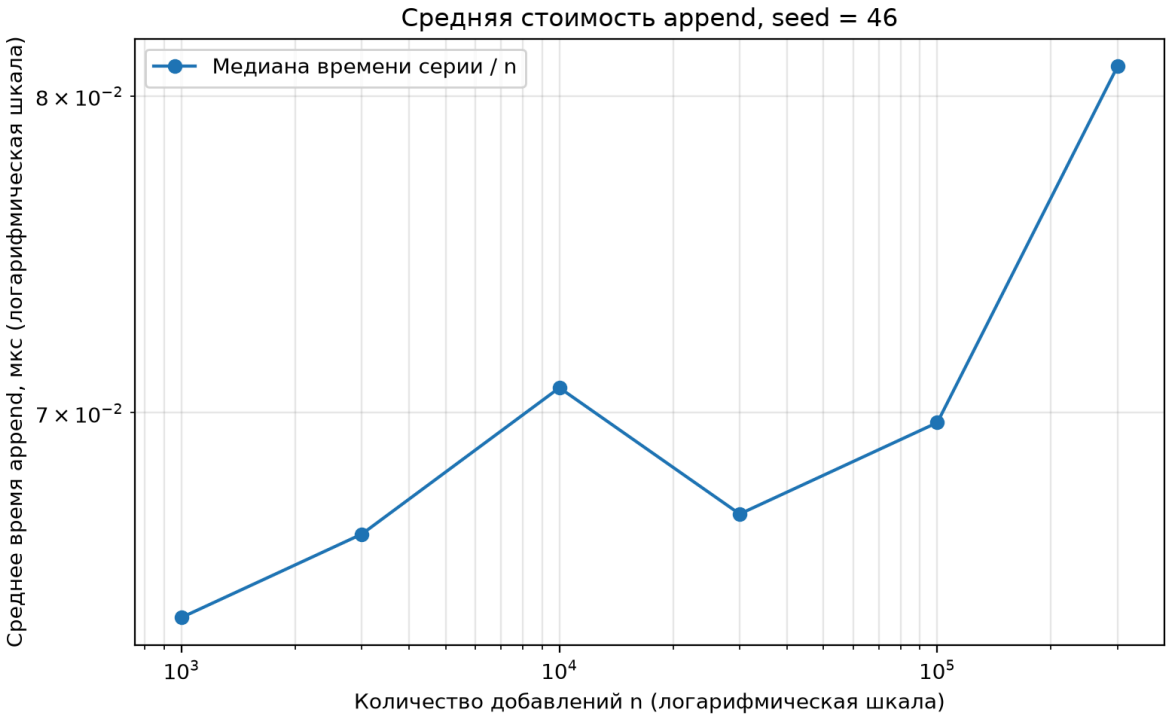


Рисунок 14 - Зависимость средней стоимости append от количества добавлений в логарифмических осях

Вывод: Массив прошёл проверки. При увеличении количества добавлений в 300 раз среднее время одного добавления изменилось примерно с 0,064 до 0,081 мкс.

Результаты согласуются с амортизированной сложностью O(1): отдельное расширение требует копирования элементов, но в среднем по длинной серии добавление остаётся быстрой операцией.

4. Задание 3

Были реализованы стек на основе динамического массива и дек на основе двусвязного списка. В стеке последний добавленный элемент извлекался первым; также была предусмотрена возможность посмотреть его без удаления. В деке были реализованы добавление и извлечение элементов с обоих концов. Для пустых структур была предусмотрена ошибка IndexError. Были подготовлены проверки порядка элементов, размера и повторного использования структур со сравнением с list и collections.deque.

```
class Stack:  1 usage  Lena *
    """Стек: последним пришёл - первым вышел (LIFO)."""

    def __init__(self) -> None:  Lena
        self._data = DynamicArray()

    def __len__(self) -> int:  Lena
        return len(self._data)

    def push(self, value) -> None:  1 usage  Lena *
        """Добавить элемент на вершину. Амортизированно O(1)."""
        self._data.append(value)

    def pop(self):  1 usage  Lena *
        """Снять и вернуть верхний элемент. O(1)."""
        if len(self._data) == 0:
            raise IndexError("Стек пуст")

        last = len(self._data) - 1
        value = self._data.get(last)
        self._data._buffer[last] = None
        self._data._size -= 1
        return value

    def peek(self):  1 usage  Lena *
        """Вернуть верхний элемент без удаления. O(1)."""
        if len(self._data) == 0:
            raise IndexError("Стек пуст")

        return self._data.get(len(self._data) - 1)
```

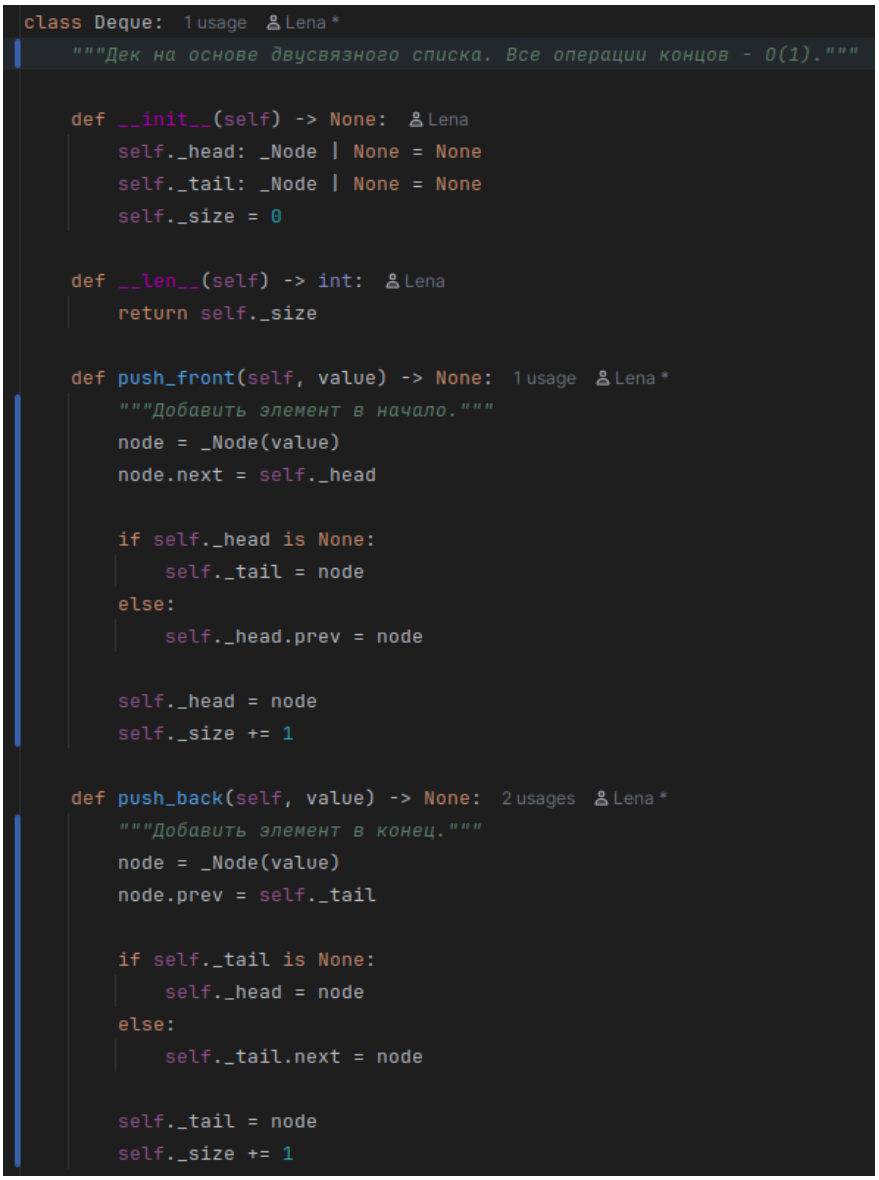
Рисунок 15 - Реализация стека на основе динамического массива

```
elena@tuf:~/Second-year-bachelor's-labs/data-structures-and-algorithms-labs$ .venv/bin/python -c "import sys; sys.path.insert(0, 'student_work/lab02'); from lab02 import Stack; s = Stack(); s.push(10); s.push(20); assert s.peek(
) == 20; assert s.pop() == 20; assert s.pop() == 10; assert len(s) == 0; print('Стек: добавили 10 и 20, извлекли 20 и 10 — OK')"
Стек: добавили 10 и 20, извлекли 20 и 10 — OK

elena@tuf:~/Second-year-bachelor's-labs/data-structures-and-algorithms-labs$ .venv/bin/python -c "import sys; sys.path.insert(0, 'student_work/lab02'); from lab02 import Stack; s = Stack(); [s.push(x) for x in range(100)]; asser
t len(s) == 100; assert s.peek() == 99; assert len(s) == 100; assert [s.pop() for _ in range(100)] == list(range(99, -1, -1)); assert len(s) == 0; print('100 добавлений и извлечений, размер и вершина — OK')"
100 добавлений и извлечений, размер и вершина — OK

elena@tuf:~/Second-year-bachelor's-labs/data-structures-and-algorithms-labs$ .venv/bin/python -c "import sys; sys.path.insert(0, 'student_work/lab02'); from lab02 import Stack; from unittest import TestCase; s = Stack(); t = Tes
tCase(); t.assertRaises(IndexError, s.pop); t.assertRaises(IndexError, s.peek); s.push(None); assert len(s) == 1; assert s.peek() is None; assert s.pop() is None; assert len(s) == 0; t.assertRaises(IndexError, s.pop); print('Пус
той стек и повторное использование — OK')"
```

Рисунок 16 - Результаты проверки работы стека



```
class Deque:  1 usage  Lena *
    """Дек на основе двусвязного списка. Все операции концов - O(1)."""

    def __init__(self) -> None:  Lena
        self._head: _Node | None = None
        self._tail: _Node | None = None
        self._size = 0

    def __len__(self) -> int:  Lena
        return self._size

    def push_front(self, value) -> None:  1 usage  Lena *
        """Добавить элемент в начало."""
        node = _Node(value)
        node.next = self._head

        if self._head is None:
            self._tail = node
        else:
            self._head.prev = node

        self._head = node
        self._size += 1

    def push_back(self, value) -> None:  2 usages  Lena *
        """Добавить элемент в конец."""
        node = _Node(value)
        node.prev = self._tail

        if self._tail is None:
            self._head = node
        else:
            self._tail.next = node

        self._tail = node
        self._size += 1
```

Рисунок 17 - Реализация добавления элементов в начало и конец дека

```python
def pop_front(self):  3 usages  Lena *
    """Извлечь элемент из начала; для пустого дека - IndexError."""
    if self._head is None:
        raise IndexError("Дек пуст")

    node = self._head
    self._head = node.next

    if self._head is None:
        self._tail = None
    else:
        self._head.prev = None

    node.next = None
    self._size -= 1
    return node.value

def pop_back(self):  1 usage  Lena *
    """Извлечь элемент из конца; для пустого дека - IndexError."""
    if self._tail is None:
        raise IndexError("Дек пуст")

    node = self._tail
    self._tail = node.prev

    if self._tail is None:
        self._head = None
    else:
        self._tail.next = None

    node.prev = None
    self._size -= 1
    return node.value
```

Рисунок 18 - Реализация извлечения элементов из начала и конца дека

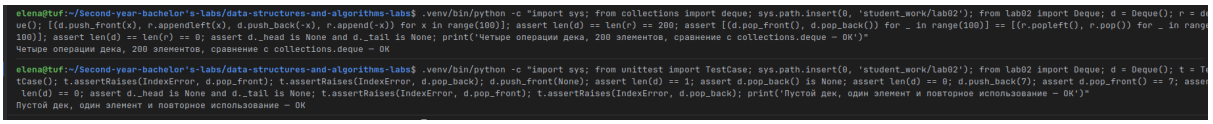


Рисунок 19 - Проверка дека: операции с обоих концов, сравнение с collections.deque и граничные случаи

Вывод: Были реализованы две структуры с разными способами доступа к элементам: стек работал с вершиной, а дек с обоими концами. Операции дека, просмотр и извлечение из стека имели сложность O(1). Добавление в стек имело амортизированную сложность O(1), поскольку иногда требовалось расширение внутреннего массива.

5. Задание 4

Была выполнена общая проверка рекурсивных функций, динамического массива, стека и дека. Результаты сравнивались с готовыми функциями и структурами Python. Проверялись содержимое и размеры структур, порядок извлечения элементов и обработка недопустимых операций.



Рисунок 20 - Результат общей проверки программы

Вывод: Общая проверка завершилась сообщением self_check: OK. В проверенных случаях результаты совпали с ожидаемыми, ошибок обнаружено не было.

6. Задание 5

Было измерено время вставки в начало обычного списка list, встроенного collections.deque и собственного дека. Для шести размеров использовались одинаковые случайные данные с seed = 46. После прогрева выполнялось пять измерений, из которых выбиралась медиана. Затем рассчитывалось среднее время одной вставки. Результаты были сохранены в таблицу и представлены на графике.

```python
def run_benchmarks(seed: int) -> None:  Lena *
    """Сравнить вставку в начало list, collections.deque и своего Deque."""
    import csv
    import json
    import platform
    from pathlib import Path

    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    def fill_list(container, values):  new *
        for value in values:
            container.insert(0, value)

    def fill_builtin_deque(container, values):  new *
        for value in values:
            container.appendleft(value)

    def fill_own_deque(container, values):  new *
        for value in values:
            container.push_front(value)

    sizes = [300, 1_000, 3_000, 10_000, 30_000, 100_000]
    repeats = 5

    # Одинаковые данные для всех трёх структур, созданные заранее
    rng = random.Random(seed)
    data = [rng.randint(-1_000_000,  b: 1_000_000) for _ in range(max(sizes))]

    cases = [
        ("list", list, fill_list),
        ("collections.deque", collections.deque, fill_builtin_deque),
        ("Свой Deque", Deque, fill_own_deque),
    ]

    cpu = "Не определён"
    for line in Path("/proc/cpuinfo").read_text(encoding="utf-8").splitlines():
        if line.startswith("model name"):
            cpu = line.split( sep: ":",  maxsplit: 1)[1].strip()
            break
```

Рисунок 21 - Код сравнения вставки в начало (часть 1)

```
print(f"\nCPU: {cpu}; Python: {platform.python_version()}")
print(f"Seed: {seed}; прогрев: 1; повторов: {repeats}; обработка: медиана")
print("\nСтруктура          |        n | Медиана серии, с | На вставку, мкс")

rows = []
costs = {name: [] for name, _, _ in cases}

for n in sizes:
    values = data[:n]

    for name, factory, fill in cases:
        # Один прогрев на свежей структуре
        container = factory()
        fill(container, values)

        times = []

        for _ in range(repeats):
            # Создание структуры выполняется до включения таймера
            container = factory()

            t0 = time.perf_counter()
            fill(container, values)
            elapsed = time.perf_counter() - t0

            times.append(elapsed)

        median_s = statistics.median(times)
        mean_us = median_s / n * 1_000_000

        rows.append([name, n, *times, median_s, mean_us])
        costs[name].append(mean_us)

        print(f"{name:<18} | {n:>8} | {median_s:>16.6f} | {mean_us:>14.6f}")

output_dir = Path(__file__).resolve().parent
```

Рисунок 22 - Код сравнения вставки в начало (часть 2)

```
csv_path = output_dir / "front_results.csv"
with csv_path.open( mode: "w", newline="", encoding="utf-8") as file:
    writer = csv.writer(file)
    writer.writerow([
        "structure", "n",
        *[f"repeat_{i}_s" for i in range(1, repeats + 1)],
        "median_total_s", "mean_insert_us",
    ])
    writer.writerows(rows)

metadata = {
    "cpu": cpu,
    "python": platform.python_version(),
    "platform": platform.platform(),
    "seed": seed,
    "sizes": sizes,
    "operation": "insert_front",
    "warmups_per_size_and_structure": 1,
    "repeats_per_size_and_structure": repeats,
    "timer": "time.perf_counter",
    "processing": "median_total_s / n * 1e6",
    "fresh_structure_each_run": True,
    "data_generated_before_timing": True,
    "total_time_unit": "seconds",
    "insert_time_unit": "microseconds",
}

metadata_path = output_dir / "front_environment.json"
metadata_path.write_text(
    json.dumps(metadata, ensure_ascii=False, indent=2) + "\n",
    encoding="utf-8",
)

fig, ax = plt.subplots(figsize=(8, 5))

for name, _, _ in cases:
    ax.loglog( *args: sizes, costs[name], "o-", label=name)

ax.set_xlabel("Количество вставок n (логарифмическая шкала)")
ax.set_ylabel("Среднее время вставки, мкс (логарифмическая шкала)")
ax.set_title(f"Сравнение вставки в начало, seed = {seed}")
ax.grid( visible: True, which="both", alpha=0.3)
ax.legend()
```

Рисунок 23 - Код сравнения вставки в начало (часть 3)

```python
ax.set_xlabel("Количество вставок n (логарифмическая шкала)")
ax.set_ylabel("Среднее время вставки, мкс (логарифмическая шкала)")
ax.set_title(f"Сравнение вставки в начало, seed = {seed}")
ax.grid( visible: True, which="both", alpha=0.3)
ax.legend()
fig.tight_layout()

graph_path = output_dir / "front_benchmark.png"
fig.savefig(graph_path, dpi=200)
plt.close(fig)

print("\nСохранены файлы:")
for path in (csv_path, metadata_path, graph_path):
    print(path)
```

Рисунок 24 - Настройка и сохранение графика

```
CPU: Intel(R) Core(TM) i7-14650HX; Python: 3.13.5
Seed: 46; прогрев: 1; повторов: 5; обработка: медиана

Структура          |        n | Медиана серии, с | На вставку, мкс
list               |      300 |         0.000010 |       0.034320
collections.deque  |      300 |         0.000003 |       0.009617
Свой Deque         |      300 |         0.000032 |       0.107313
list               |     1000 |         0.000063 |       0.063412
collections.deque  |     1000 |         0.000011 |       0.010518
Свой Deque         |     1000 |         0.000100 |       0.100399
list               |     3000 |         0.000435 |       0.145067
collections.deque  |     3000 |         0.000031 |       0.010355
Свой Deque         |     3000 |         0.000393 |       0.131070
list               |    10000 |         0.005376 |       0.537599
collections.deque  |    10000 |         0.000095 |       0.009481
Свой Deque         |    10000 |         0.001175 |       0.117480
list               |    30000 |         0.050322 |       1.677403
collections.deque  |    30000 |         0.000290 |       0.009673
Свой Deque         |    30000 |         0.004194 |       0.139806
list               |   100000 |         0.558255 |       5.582553
collections.deque  |   100000 |         0.000930 |       0.009299
Свой Deque         |   100000 |         0.014800 |       0.147999

Сохранены файлы:
/home/elena/Second-year-bachelor's-labs/data-structures-and-algorithms-labs/student_work/lab02/front_results.csv
/home/elena/Second-year-bachelor's-labs/data-structures-and-algorithms-labs/student_work/lab02/front_environment.json
/home/elena/Second-year-bachelor's-labs/data-structures-and-algorithms-labs/student_work/lab02/front_benchmark.png
```

Рисунок 25 - Результаты замеров для списка и двух деков

```
{
  "cpu": "Intel(R) Core(TM) i7-14650HX",
  "python": "3.13.5",
  "platform": "Linux-6.12.111+deb13-amd64-x86_64-with-glibc2.41",
  "seed": 46,
  "sizes": [
    300,
    1000,
    3000,
    10000,
    30000,
    100000
  ],
  "operation": "insert_front",
  "warmups_per_size_and_structure": 1,
  "repeats_per_size_and_structure": 5,
  "timer": "time.perf_counter",
  "processing": "median_total_s / n * 1e6",
  "fresh_structure_each_run": true,
  "data_generated_before_timing": true,
  "total_time_unit": "seconds",
  "insert_time_unit": "microseconds"
}
```


Рисунок 26 - Условия проведения замеров

	structure	n	repeat_1_s	repeat_2_s	repeat_3_s	repeat_4_s	repeat_5_s	median_total_s	mean_insert_u
1	list	300	1.0759998986031 86e-05	1.0341002052882686e-05	1.02960002550389 62e-05	1.0038998880190775e-05	9.6490002761129 29e-06	1.0296000255038962e-05	0.034320000
2	collections.deque	300	3.0639967008028 18e-06	2.8899994504172355e-06	2.8290014597587 287e-06	2.8499998733432 62e-06	2.8850008675362 915e-06	2.8850008675362915e-06	0.009616669
3	Свой Deque	300	3.2280000596074 39e-05	3.2193998777074 74e-05	3.5884000681107 864e-05	2.9384998924797 4e-05	2.9951999749755 487e-05	3.2193998777074 74e-05	0.107313
4	list	1000	6.4583000494167 21e-05	6.3411996961804 11e-05	5.9545000112848 356e-05	6.2940998759586 36e-05	6.3494997448287 9e-05	6.3411996961804 11e-05	0.063411
5	collections.deque	1000	1.0518000635784 12e-05	1.0806001228047 535e-05	1.0436000593472 272e-05	1.0570001904852 688e-05	1.0285999451298 267e-05	1.0518000635784 12e-05	0.010518000
6	Свой Deque	1000	9.1164998593748 17e-05	0.0001533559989 1841412	0.0001003990007 541291	0.00014576800094801	9.8283002444877 28e-05	0.0001003990007 541291	0.100399000
7	list	3000	0.0004352010000 7299334	0.0004567819996 7369437	0.0004314310026 529711	0.0004311820011 935197	0.0004380879981 908947	0.0004352010000 7299334	0.145067000
8	collections.deque	3000	2.962800135719 59e-05	2.9652001103386 283e-05	3.1063998903846 37e-05	3.1087998650036 75e-05	3.1555999157717 45e-05	3.1063998903846 37e-05	0.010354666
9	Свой Deque	3000	0.0004073230011 272244	0.0003932100007 6597556	0.0003694150000 2285466	0.0015399209987 663198	0.0003845560022 4551745	0.0003932100007 6597556	0.131070000
10	list	10000	0.0055629130001 76195	0.0053759949987 5796	0.0054170689982 12926	0.0053724130011 68752	0.0052838920018 98494	0.0053759949987 5796	0.537599
11	collections.deque	10000	9.4625000201631 34e-05	9.4727998657617 72e-05	9.4805000117048 62e-05	0.0001017020003 9283372	0.0001031190004 1415356	9.4805000117048 62e-05	0.009480500
12	Свой Deque	10000	0.0019040569000 576079	0.0011748030010 494404	0.0010353290017 519612	0.0018997160004 801117	0.0011261020008 532796	0.0011748030010 494404	0.117480300
13	list	30000	0.0503457679988 0597	0.0506657429978 0768	0.0503220940008 7595	0.0502213499967 18345	0.0502287189992 785	0.0503220940008 7595	1.677403
14	collections.deque	30000	0.0002978449992 951937	0.0002843719994 4164604	0.0002865969819 399975	0.0002967330001 411028	0.0002981890002 2492744	0.0002981890002 2492744	0.009672966
15	Свой Deque	30000	0.0190252940010 39583	0.0036396079995 02914	0.0040899770028 77276	0.0041941939998 65092	0.0047462480033 57323	0.0041941939998 65092	0.139806460
16	list	100000	0.5591855710008 531	0.5596747649979 879	0.5579474870028 207	0.5582553280000 866	0.5581729689840 58	0.5582553280000 866	5.582553
17	collections.deque	100000	0.0009370080006 192438	0.0009309589986 514766	0.0009229870011 040475	0.0009209980016 748887	0.0009298739969 381131	0.0009298739969 381131	0.009298739
18	Свой Deque	100000	0.039681341997 49329	0.0140428459999 38436	0.0141269670020 84712	0.0444603020005 16094	0.0147998549982 73076	0.0147998549982 73076	0.147998549

Рисунок 27 - Таблица результатов замеров

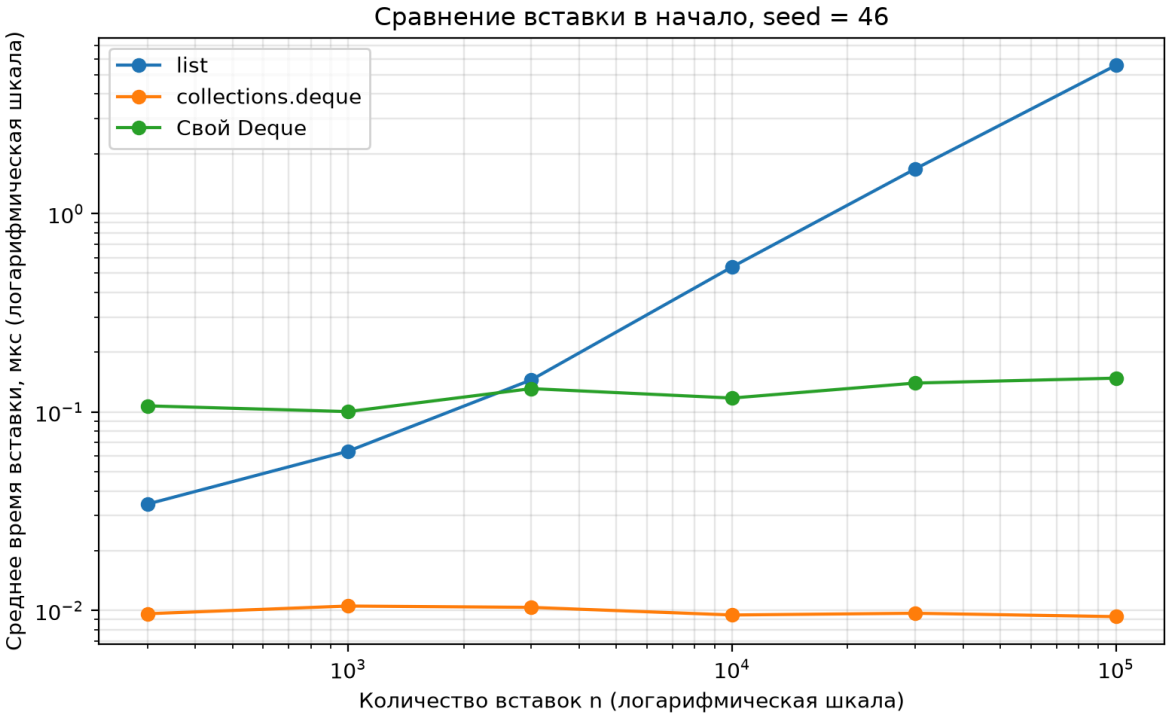


Рисунок 28 - Сравнение времени вставки в начало списка и двух деков

Вывод: При увеличении количества элементов вставка в начало списка замедлялась из-за сдвига уже добавленных элементов. У обоих деков время одной вставки изменялось значительно меньше, что согласуется со сложностью O(1). Самым быстрым оказался встроенный дек: на 100 000 вставок ему потребовалось около 0,00093 с, нашему деку - 0,0148 с, а списку - 0,558 с.

7. Задание 6

Для анализа использовался банковский метод. Запись нового элемента и копирование одного элемента считались одной единицей работы. Каждому добавлению назначалась условная стоимость 3 единицы: одна тратилась на запись, две откладывались на будущее расширение. Начальная ёмкость массива равнялась 4. Первые четыре добавления накапливали 8 единиц. При пятом добавлении требовалось скопировать 4 элемента, поэтому накоплений хватало. Далее при заполнении второй половины массива ёмкости 2C выполнялись C добавлений. Они накапливали 2C единиц достаточно для копирования всех 2C элементов при следующем расширении. Таким образом, кредит покрывал каждое копирование.

Вывод: Отдельное добавление при расширении занимает O(n), поскольку приходится копировать элементы. Но на последовательность из n добавлений требуется суммарно O(n) работы. Поэтому амортизированная стоимость одного добавления O(1).

Заключение

В ходе лабораторной работы были реализованы рекурсивные функции, динамический массив, стек и дек. Проверки показали правильную работу в рассмотренных случаях. Мемоизация сократила количество вызовов при вычислении чисел Фибоначчи. Замеры показали, что при больших объёмах данных вставка в начало деков быстрее, чем в начало списка. Банковским методом было объяснено, почему добавление в динамический массив имеет амортизированную сложность O(1).